

Arquitectura de Microservicios con Contenedores

Docentes

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Introducción a las Arquitectura de Microservicios

Objetivos de la clase

Al finalizar esta clase vas a poder:

- Explicar el origen y la motivación de la arquitectura de microservicios.
- Diferenciar microservicios de arquitecturas monolíticas y de SOA.
- Aplicar los criterios de **bajo acoplamiento** y **alta cohesión** para definir los límites de un servicio.
- Reconocer cuándo microservicios **no** es la mejor opción para un negocio.
- Describir los módulos que componen la anatomía genérica de un microservicio.

Inicios

En 2014 Martin Fowler & James Lewis escriben un artículo donde plantean los problemas de las arquitecturas de software existentes y por qué hace falta redefinir las metodologías de trabajo.

- Conceptualmente plantea que una arquitectura basada en microservicios no es más que un conjunto de servicios desarrollados en forma independientes, cada uno ejecutándose en forma independiente, con sistemas de comunicación simples entre ellos.
- Los servicios deben poder ser ejecutados en forma independiente, pero coordinada.

Inicios

- Anteriormente las arquitectura seguían un modelo más monolítico, donde toda la aplicación se ejecutaba en un sólo servidor, o a lo sumo un modelo de dos o tres capas.
- El escalamiento, normalmente era vertical, poniendo un servidor más potente, u horizontal, pero replicando la misma aplicación completa varias veces.
- El planteo de los microservicios es dividir en servicios independientes las partes que componen una aplicación, de modo hacer elementos funcionales independientes.

Early adopters de Microservicios

Amazon fue uno de los early adopters de una arquitectura de microservicios, y la justificación de su CTO fue:

- Podemos escalar cada servicios en forma independiente
- Mantener en forma paralela la disponibilidad de cada servicio
- Facilidad de agregar un servicio nuevo sin la necesidad de una reconfiguración masiva de todo el sistema.



Fuente: <https://queue.acm.org/detail.cfm?id=1142065>

Early adopters de Microservicios

Las tiendas Gilt habían adoptado microservicios en el 2015 y la justificación que dio su VP de ingeniería fue:

- Mayor independencia entre equipos de trabajo, que resultaban en mayor velocidad de delivery
- Permite muchas iniciativas en paralelo
- Soporta múltiples tecnologías, lenguajes y frameworks distintos
- Permite limitar mejor el fallo de un servicio.
- Facilita la innovación por medio del concepto de disposable code.

The logo for GILT, featuring the word "GILT" in a large, bold, black serif font.

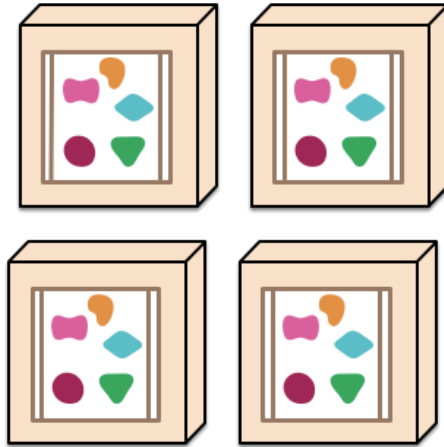
Fuente: <https://www.infoq.com/news/2015/04/scaling-microservices-gilt/>

Monolítico vs Microservicios

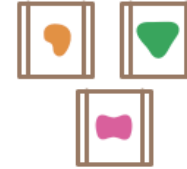
A monolithic application puts all its functionality into a single process...



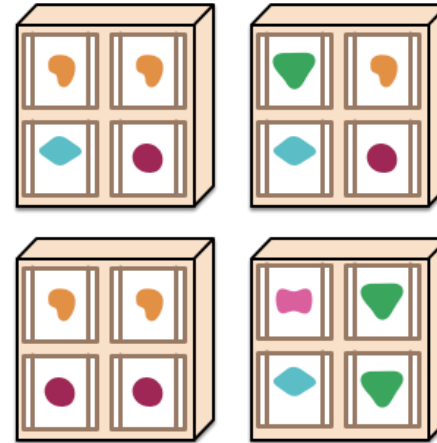
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.



Fuente: <https://martinfowler.com/articles/microservices.html>

¿Y no era esto lo mismo que SOA?

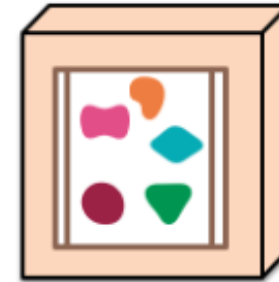
SOA también propone dividir en servicios. La diferencia está en el **cómo**:

	SOA	Microservicios
Comunicación	Middleware centralizado (ESB)	Directa y simple (HTTP, mensajería)
Gobernanza	Centralizada (SOAP, WS-*)	Descentralizada, cada equipo elige
Datos	Base de datos compartida	Cada servicio con su propia base
Tamaño	Servicios grandes, por sistema	Pequeños, por capacidad de negocio
Equipos	Un equipo, varios servicios	Un equipo dueño de punta a punta

El ESB de SOA termina siendo un monolito disfrazado: toda la lógica de integración en un solo lugar.

Problemas Monolíticos

- El costo de mantenimiento de las aplicaciones monolíticas es muy alto.
- Un pequeño cambio requiere volver a construir toda la aplicación.
- El tiempo de despliegue suele ser alto, de modo que las afectaciones de servicio llevan a un despliegue menos frecuente.
- El escalamiento se realiza replicando todos los componentes de la solución, y no solamente aquellos que tienen más carga.



Soluciones de los Microservicios

- En vez de una aplicación por servidor, múltiples servicios, más simples, en uno o más servidores.
- Interfaces muy bien definidas, de modo que cualquier cambio interno, que mantenga la interfaz, puede realizarse sin que el resto de los servicios se enteren.
- La independencia en la implementación permite usar para cada servicio un lenguaje independiente, o un estilo de base de datos diferente, sin afectar a otras partes de la aplicación.
- Este modelo, no fue original, ya que UNIX fue diseñado con el mismo principio, pero no se había aplicado al desarrollo de aplicaciones.

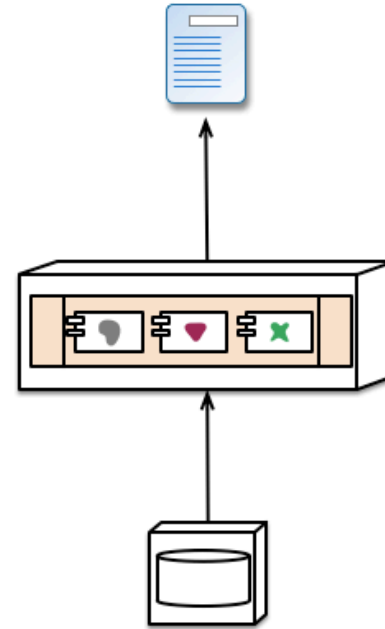
Microservicios: Separación en servicios

- Un componente es una unidad de software que puede ser reemplazada o actualizada en forma independiente. Cada componente implementa un servicio que forma parte de la aplicación.
- Un servicio no es una librería. Las librerías son software bien separado y con una interfaz bien definida que se ejecuta dentro del mismo proceso, compartiendo memoria y CPU.
- Un servicio puede reemplazarse y el programa que consume esos servicios no se entera. Si cambiamos una librería es necesario reiniciar el programa o en algunos casos hasta compilarlo de nuevo.

Modelo tradicional

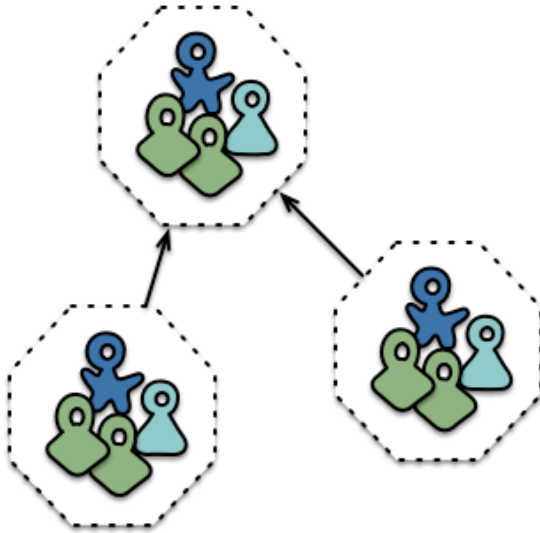


Siloed functional teams...

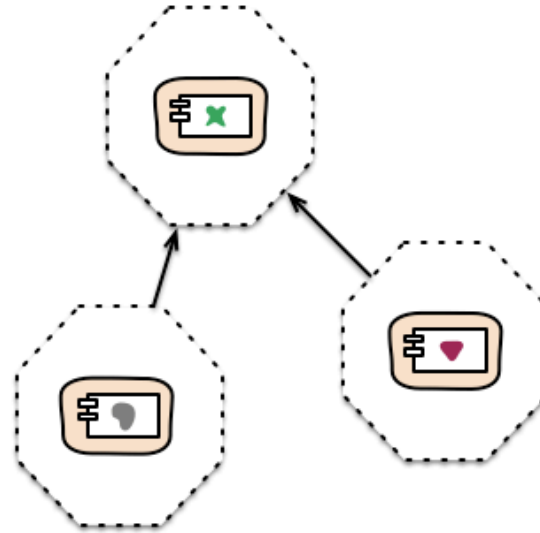


... lead to siloed application architectures.
Because Conway's Law

Equipos multifuncionales por servicio



Cross-functional teams...



... organised around capabilities
Because Conway's Law

Fuente: <https://martinfowler.com/articles/microservices.html>

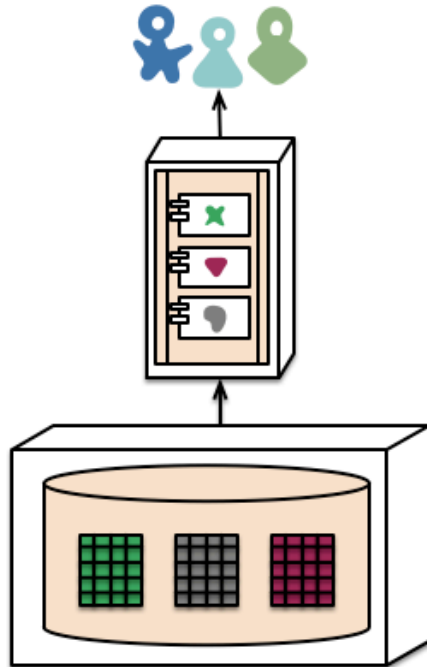
Microservicios: Productos, no Proyectos

- El modelo de desarrollo basado en proyectos define que para un proyecto se arma un equipo de trabajo, hace el diseño del cambio, lo desarrolla, lo prueba, implementa y luego se disuelve el equipo.
- El modelo orientado a producto, tiene un equipo de trabajo interdisciplinario que maneja todos los aspectos necesarios para el desarrollo del producto. Toman responsabilidad por el producto y no por las funcionalidades.

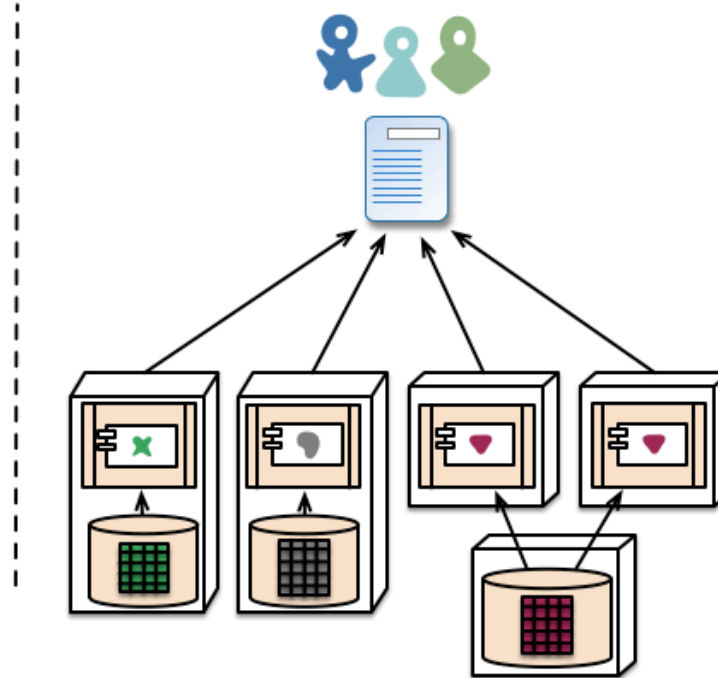
The traditional model is that you take your software to the wall that separates development and operations, and throw it over and then forget about it. Not at Amazon. You build it, you run it. This brings developers into contact with the day-to-day operation of their software.

– Werner Vogels, Amazon CTO, 2006

Microservicios: Descentralización de las DB



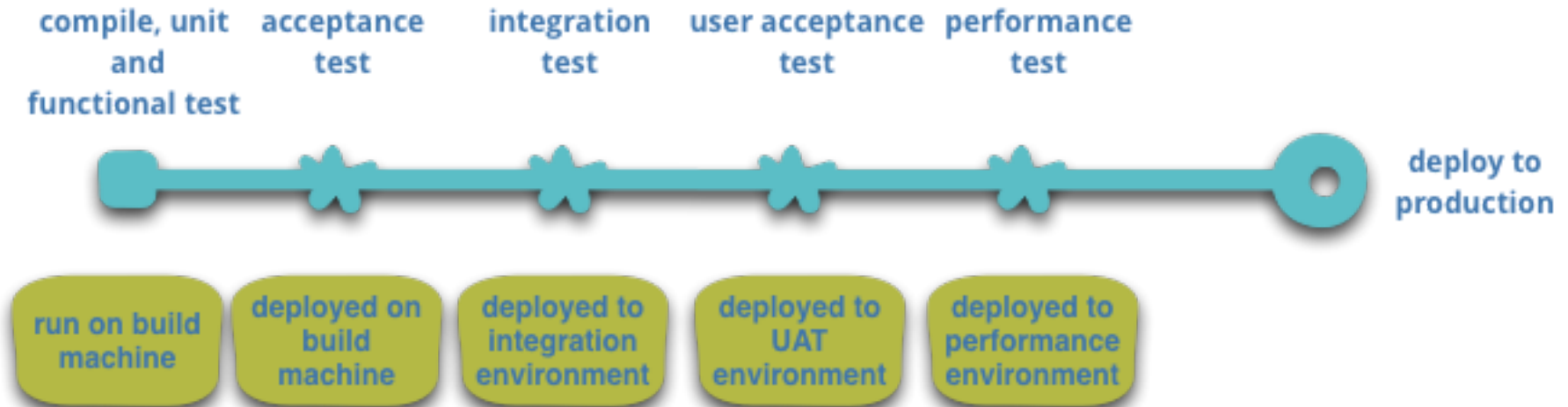
monolith - single database



microservices - application databases

Fuente: <https://martinfowler.com/articles/microservices.html>

Microservicios: Automatización (CI/CD)



Fuente: <https://martinfowler.com/articles/microservices.html>

Microservicios: Pros/Cons

Pros

-  Strong Module Boundaries
-  Independent Deployment
-  Technology Diversity

Cons

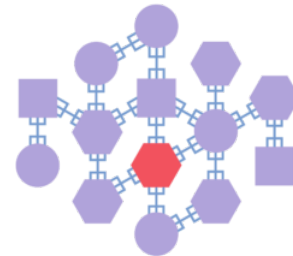
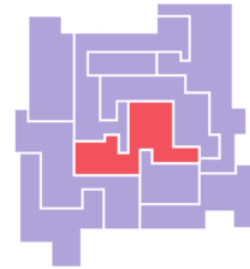
-  Distribution
-  Eventual Consistency
-  Operational Complexity

Criteriaos Básicos de diseño

Principios universales en desarrollo

Bajo acoplamiento:

- El acoplamiento se define como la dependencia que tiene un servicio de otro. El cambio en un servicio no depende del cambio en otro servicio.
- Un sistema de intercambio de mensajes que requiera que para agregar un campo más en un mensaje se debe cambiar a todos los posibles receptores, tiene alto acoplamiento.
- Un sistema que no depende de la estructura del mensaje y donde todos los posibles receptores puedan ignorar los campos que no necesitan, tiene bajo acoplamiento.



Un ejemplo concreto de acoplamiento

Un servicio de **Pedidos** publica un evento cuando se confirma una compra. Queremos agregar un campo `couponCode`.

Alto acoplamiento: el mensaje es un contrato rígido (ej. un schema estricto sin campos opcionales). Cada consumidor debe recompilar y redesplegar antes de que el productor pueda publicar el nuevo campo.

Bajo acoplamiento: el consumidor ignora los campos que no conoce. El productor agrega `couponCode` y despliega solo; nadie más se entera hasta que decida usarlo.

```
// Evento OrderConfirmed
{
  "orderId": "A-1029",
  "customerId": "8841",
  "total": 4200.00,
  "couponCode": "ITBA10" // nuevo campo
}
```

Los consumidores que solo leen `orderId` y `total` **no se rompen** con el campo nuevo: esa tolerancia es lo que baja el acoplamiento.

Principios universales en desarrollo

Alta cohesión

- Se denomina la cohesión como qué tan relacionadas están todas las tareas de un servicio para con el objetivo específico de ese servicio.
- Una alta cohesión simplifica el mantenimiento de un servicio, ya que todo lo que necesito para ese servicio se encuentra en un mismo lugar.
- Dentro de Microservicios este concepto es clave para asegurar la facilidad para actualizar los servicios en forma continua e independiente.

We want to design components that are self-contained: independent, and with a single, well-defined purpose

When you come across a problem, assess how localized the fix is. Do you change just one module, or are the changes scattered throughout the entire system? When you make a change, does it fix everything, or do other problems mysteriously arise?

– The Pragmatic Programmer

Fuente: <https://pragprog.com/titles/tpp20/the-pragmatic-programmer-20th-anniversary-edition/>

Still SOLID...

Robert C. Martin planteó el criterio de diseño SOLID para una arquitectura OO:

- **Single Responsibility Principle:** Cada servicio se ocupa de una sólo función.
- **Open-Close Principle:** Cada servicio que requiera una modificación debe ser extendido, no re hecho.
- **Liskov Substitution Principle:** Este principio habla del concepto de herencia entre objetos, el rol padre-hijo entre objetos y que la interfaz de padre es la que debe aplicarse a cualquier reemplazo.
- **Interface Segregation Principle:** Mantener las APIs simples para que cada cliente pueda usar la parte que necesite.
- **Dependency Inversion Principle:** Depender de abstracciones, no de los detalles.

Fuente:
https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf

La ley de Conway

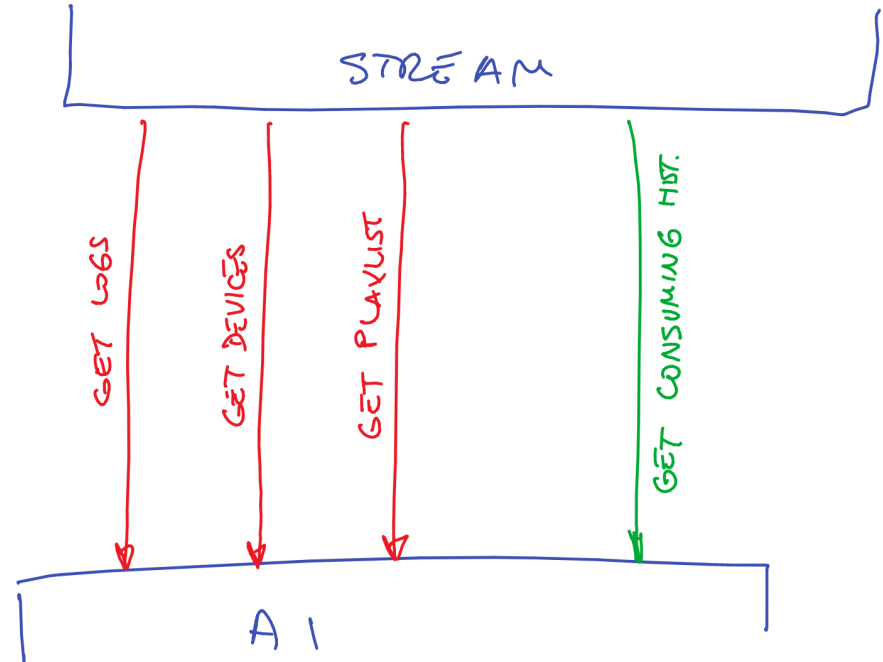
- Melvin Conway dijo en 1967 que: los sistemas tienden a organizarse de la misma manera que se definen los flujos de comunicación entre áreas funcionales en las empresas.
- Los dominios que mencionamos antes tienden a seguir este mismo principio.

Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.

– Melvin E. Conway

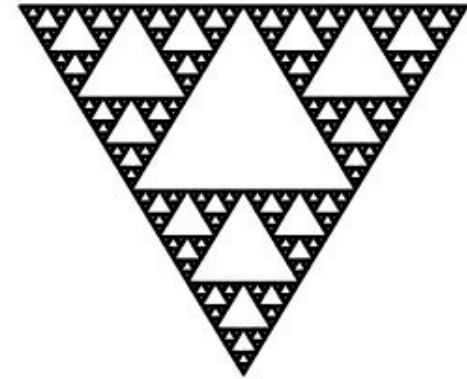
Business Capabilities

- Si hablamos de definir un contrato entre contextos, es importante no limitar ese contrato a solamente el intercambio de datos (operaciones CRUD).
- Debo hacerme la pregunta: ¿Qué capacidades tiene ese contexto y quiero que me brinde?
- Al pedir capacidades y no datos, estos bajando el **acoplamiento** y aumentando la **cohesión** para ese contexto.



Límites y más límites

- Cada vez que se define un contexto, podemos mirar dentro del contexto y ver que los modelos también tiene interfaces con otros modelos, de modo que podríamos generar sub contextos y volver separar estos modelos en sus propios dominios.
- Un servicio debería implementar pocos requerimientos funcionales. Si tiene muchos, probablemente deba dividirse.



Otros métodos de descomposición

- Recién vimos el método de descomposición por contextos, pero existen otros métodos de descomposición:
 - Por funcionalidad
 - Por madurez
 - Por patrones de acceso a los datos
- Funcionalidad y madurez son más usados en procesos de refactoring de aplicaciones monolíticas.



No siempre se puede con Microservicios

Sólo importa el valor al negocio

- Todos los beneficios que vimos en los casos no se implementaron para hacer aplicaciones más robustas, más dinámicas o más estables.
- La única justificación para su implementación es aportar valor al negocio.
 - Unos negocios necesitan aplicaciones que se actualicen diariamente (Rappi). Otras, un release muy estable cada varios meses (Leistung).
 - Algunas empresas operan muy distribuidas (Waze) y otras muy centralizadas (Retail físico).
 - Equipos de trabajo chicos pueden trabajar mejor bajo un mismo alineamiento, que equipos muy grandes, donde un liderazgo distribuido puede ser una mejor solución.
- Los microservicios son un medio más para el diseño de sistemas adecuados para el negocio, no un fin en sí mismos.

Para discutir en clase

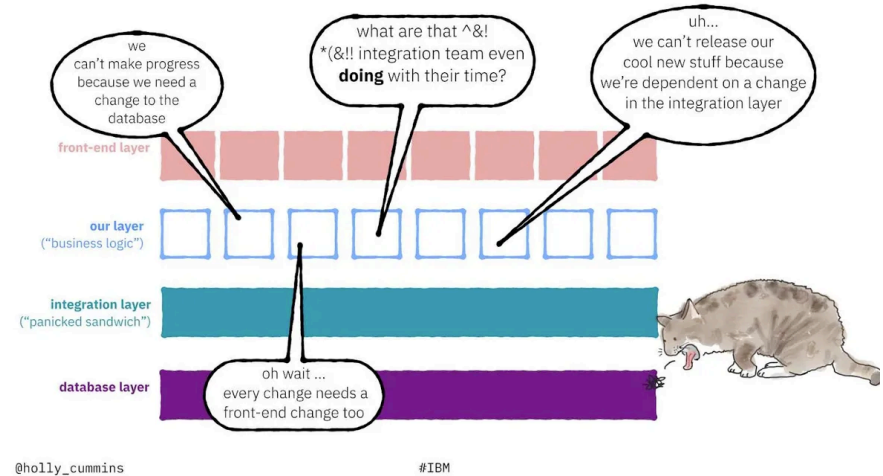
Para cada caso, decidí: **¿monolito o microservicios?** y justificá con al menos uno de los criterios vistos (equipo, frecuencia de release, escalamiento, distribución del negocio).

Caso	¿Monolito o microservicios?
Una fintech que lanza features nuevas todas las semanas con 6 equipos independientes	?
Un sistema de facturación interno, un solo equipo de 4 personas, cambia 2 veces por año	?
Una app de e-commerce donde el catálogo recibe 100x más tráfico que el checkout	?
Una startup de 3 personas construyendo su primer MVP	?

No hay una única respuesta correcta: lo importante es el criterio usado para justificarla.

Fallos al implementar Microservicios

- Se deben implementar porque su naturaleza se alinea mejor con los requerimientos del negocio, no porque está de moda o es una tecnología cool.



Fuente: <https://www.infoq.com/articles/microservices-seven-fail/>

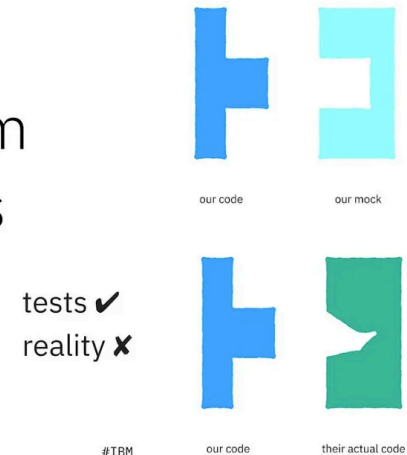
Fallos al implementar Microservicios

- Las aplicaciones monolíticas distribuidas no son microservicios.
- Distribuido no necesariamente significa desacoplado.
- Muchos microservicios operando sobre el mismo dominio de datos. La independencia de cada microservicios debe ser lo más completa posible (front, backend y datos) para asegurar su éxito.
- Sistemas real-time o donde la coordinación es crítica, no son buenos casos de éxito para los microservicios.

Fallos al implementar Microservicios (cont.)

- El contrato (acuerdo) entre el proveedor y el consumidor. Cómo se define la interfaz y cómo se generan los procesos de prueba para el consumidor y el proveedor de un servicio. La capacidad de coordinar en el diseño entre equipos es crítica.
- Este problema se resuelve con **contract testing** (ej. Pact): cada consumidor define qué espera del proveedor, y esas expectativas se verifican automáticamente en el pipeline de CI de ambos lados, sin necesidad de un entorno integrado end-to-end.

the problem
with mocks



@holly_cummins

#IBM

Fallos al implementar Microservicios (cont.)

- La velocidad debe ser consistente entre equipos para poder aprovechar la frecuencia y metodología de despliegue de los Microservicios.

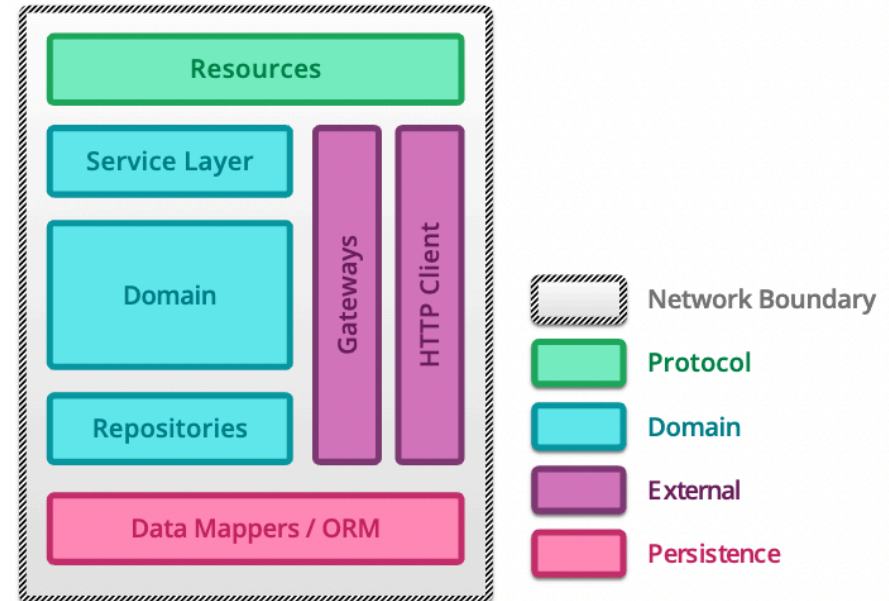


Fuente: <https://www.infoq.com/articles/microservices-seven-fail/>

Framework genérico de un microservicio

Framework genérico

Este es un framework genérico que se adapta a la gran mayoría de microservicios que se podrían implementar.

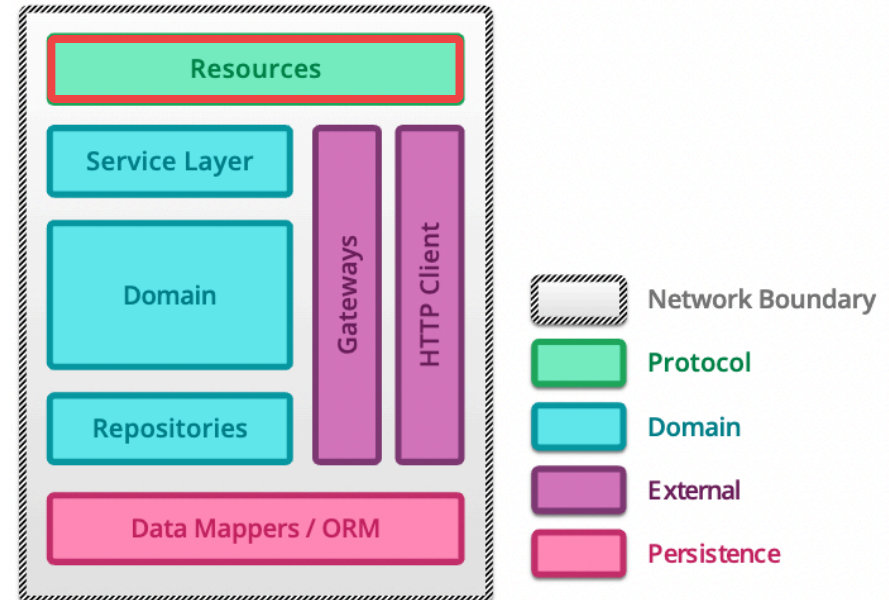


Fuente: <https://martinfowler.com/articles/microservice-testing/#anatomy-modules>

Framework genérico

Recursos:

- Actúan como interfaz entre los requerimientos recibidos por el protocolo externo y el mensaje a los objetos internos del servicio.
- Validan parámetros y dan formato a las respuestas.

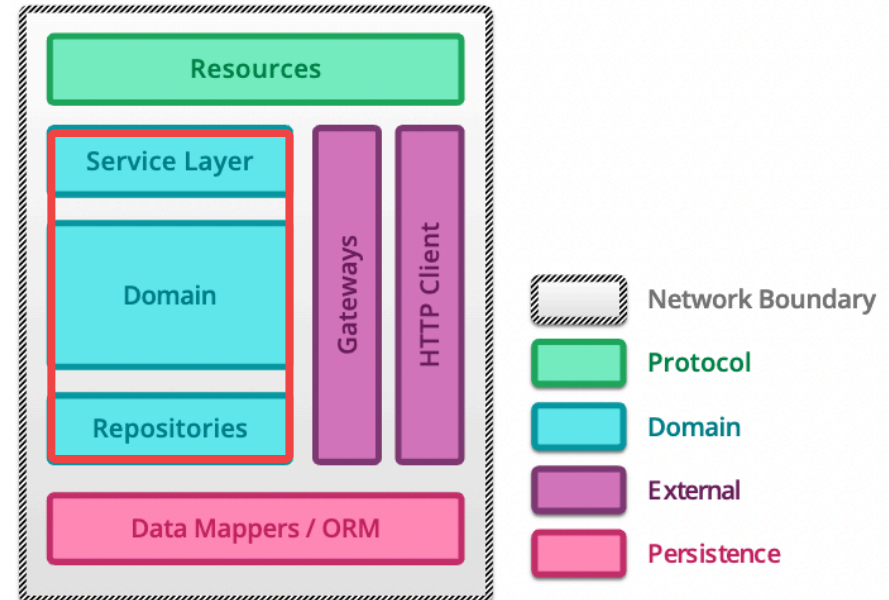


Fuente: <https://martinfowler.com/articles/microservice-testing/#anatomy-modules>

Framework genérico

Domain Model:

- Implementa lógica de negocio.
- El service layer coordina actividades complejas que requieren mantener un estado.
- Los repositories gestionan la persistencia de los datos

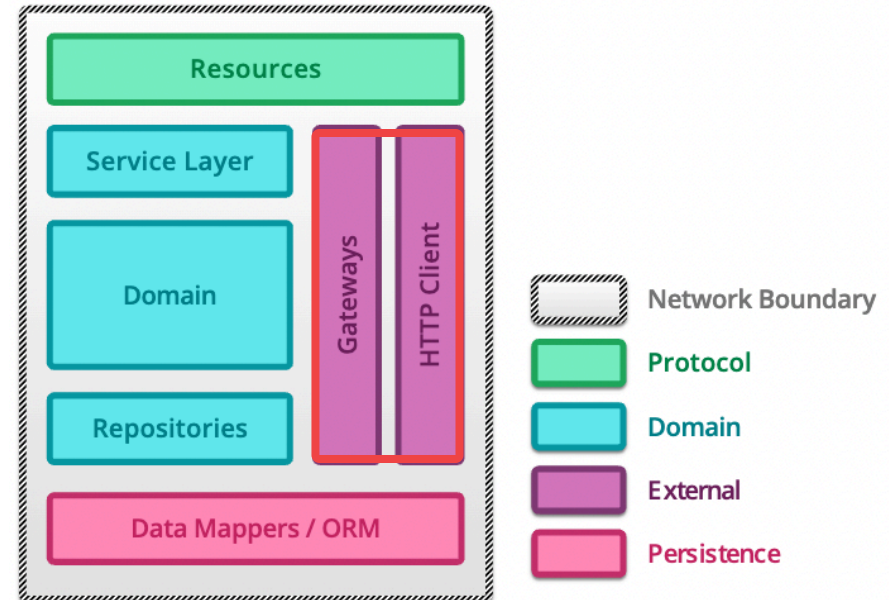


Fuente: <https://martinfowler.com/articles/microservice-testing/#anatomy-modules>

Framework genérico

Gateway:

- Gestiona la lógica para las comunicaciones salientes hacia otros servicios.
- Sincronismo, encap/decap, autenticación.
- Gestión de errores

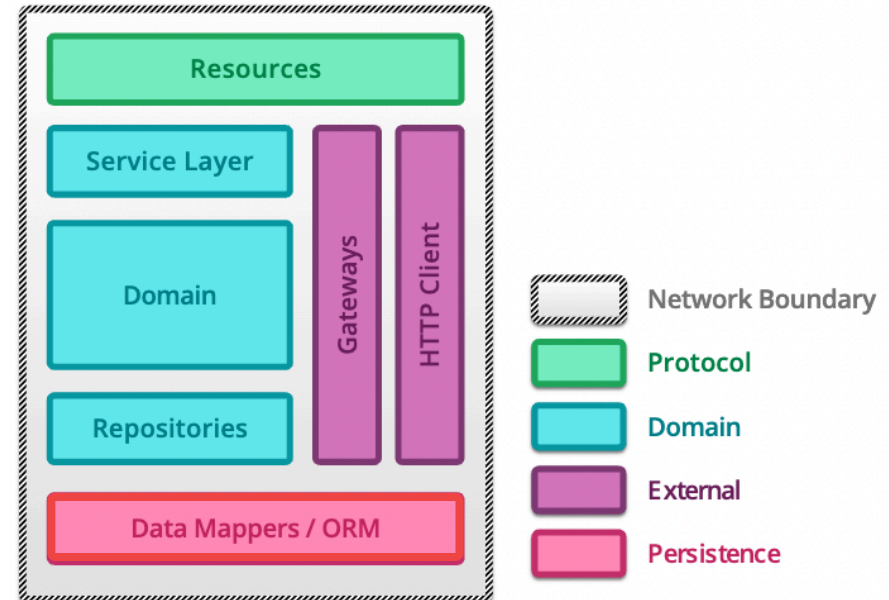


Fuente: <https://martinfowler.com/articles/microservice-testing/#anatomy-modules>

Framework genérico

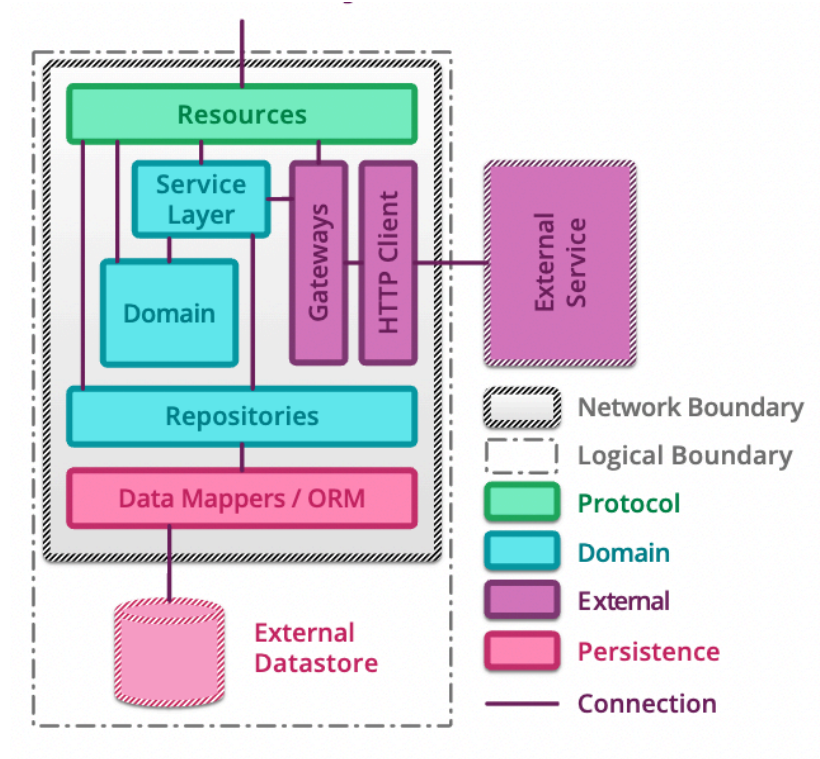
Data Mapper/ORM:

- Object relation mapping
- Mantiene la persistencia sobre los objetos del dominio.
- Hace de interfaz entre el data store y los objetos del servicio.



Fuente: <https://martinfowler.com/articles/microservice-testing/#anatomy-modules>

Framework genérico



Fuente: <https://martinfowler.com/articles/microservice-testing/#anatomy-modules>

Repaso: lo que nos tenemos que llevar

- Microservicios = servicios independientes, con interfaces bien definidas, desplegables y escalables por separado.
- El objetivo nunca es la tecnología: es aportar **valor al negocio** (velocidad de entrega, escalamiento selectivo, autonomía de equipos).
- Los criterios de diseño clave son **bajo acoplamiento** y **alta cohesión**; SOLID, la ley de Conway y los bounded contexts son herramientas para lograrlos.
- No siempre es la mejor opción: monolitos distribuidos, dominios de datos compartidos y sistemas real-time son señales de alerta.
- Todo microservicio comparte una anatomía común: Resources, Domain Model, Gateways y Data Mappers/ORM.

Glosario

- **Acoplamiento:** dependencia entre servicios.
- **Cohesión:** qué tan alineadas están las tareas de un servicio con su propósito.
- **Bounded Context:** límite dentro del cual un modelo de dominio es válido y consistente.
- **Contract testing:** verificar el contrato entre consumidor y proveedor sin un entorno integrado.
- **Ley de Conway:** la arquitectura del sistema refleja la estructura de comunicación de la organización.
- **Monolito distribuido:** servicios separados en el despliegue pero acoplados en los datos o en el diseño.
- **Business capability:** lo que un contexto puede *hacer* por otros, no solo los datos que expone.

Fin!!